

Evolving Super Mario Bros Controllers via Typed Genetic Programming

Miguel Cabral Pinto and Tiago Silva

Departamento de Engenharia Informática, Universidade de Coimbra
`{mcp, tds}@student.uc.pt`

Abstract. We evolve game controllers for Super Mario Bros using Genetic Programming, pursuing two successive objectives: completing levels and maximizing enemy kills and coins obtained. Each individual is a typed syntax tree that compiles to an executable Python function, so the search operates over the space of structured programs. We define a typed grammar giving agents access to a 22×22 grid of terrain and enemy data, and train through two goal stages: level completion and kill/coin maximization. Bloat is controlled through Double Tournament selection with parsimony pressure; generalization is promoted through a rotating seed pool that changes every 10 generations. **[TODO: 2 sentences: win rate achieved by the evolved agent, improvement over the random baseline.]**

1 Introduction

In this work we use Genetic Programming (GP) [2] to evolve controllers for the Mario AI framework [5]. GP is well-matched to this problem for two reasons: it requires only a scalar fitness signal and no gradient information, and it produces programs as output, meaning the evolved controller is a readable Python function. Since the failure mode of any agent is visible in the code, iterating on the fitness function is fast and targeted.

The course provided a base GP repository implemented with DEAP [1]: a minimal random search over a small typed grammar with two senses (`on_ground`, `can_jump`), two actions (`RIGHT`, `JUMP`), and basic `IF/SEQ` program structure. During our work, we expanded the grammar to give agents better perception of terrain and enemies, designed the fitness function, and replaced the random search with a full evolutionary algorithm.

Training follows two stages: a completion stage, where the goal is to reach the end of the level, and a hunter stage, where additional rewards are awarded for killing enemies. The fitness function went through several iterations; the choices behind the final version are described in Section 2.3.

2 Methodology

2.1 Environment

The Mario AI framework [5] is a Java simulator that exposes a socket API through which external agents send action vectors and receive observations at a fixed step rate. At each step the agent receives information about the simulation (e.g., Mario state & position, level scene observation grid).

The agent outputs a boolean vector [LEFT, RIGHT, DOWN, JUMP, SPEED] at each step, controlling Mario’s actions. Difficulty is controlled along two independent axes: mario mode (small, large, or fire) and level difficulty, which scales obstacle density and enemy variety.

2.2 Typed Grammar and Representation

Each individual is a program tree in a typed grammar built on top of the provided DEAP [1] scaffold. We extended the original four types (Cond, Key, Bool, Expr) to nine: Program, Stmt, Cond, Comparator, Key, Bool, Offset, EnemyKind, and TileValue.

Program structure. A **Program** is a non-empty sequence of statements built left-recursively:

$$\text{Program} \rightarrow \text{END}(\text{Stmt}) \mid \text{SEQ}(\text{Stmt}, \text{Program})$$

Statements. A **Stmt** is a conditional branch or a direct key assignment:

$$\text{Stmt} \rightarrow \text{IF}(\text{Cond}, \text{Stmt}) \mid \text{IF_ELSE}(\text{Cond}, \text{Stmt}, \text{Stmt}) \mid \text{SET_ACTION}(\text{Key}, \text{Bool})$$

Conditions. Conditions combine two perception predicates with standard boolean connectives:

$$\begin{aligned} \text{Cond} \rightarrow & \text{AND}(\text{Cond}, \text{Cond}) \mid \text{OR}(\text{Cond}, \text{Cond}) \mid \text{NOT}(\text{Cond}) \\ & \mid \text{CheckEnemyAt}(\text{Offset}_x, \text{Offset}_y, \text{Comparator}, \text{EnemyKind}) \\ & \mid \text{CheckLandscapeAt}(\text{Offset}_x, \text{Offset}_y, \text{Comparator}, \text{TileValue}) \\ & \mid \text{IsMarioOnGround} \mid \text{MayMarioJump} \end{aligned}$$

Both predicates access the 22×22 grid at offset $(\Delta x, \Delta y)$ relative to Mario’s cell. Horizontal offsets $\Delta x \in \{0, 1, 2, 3\}$ let the agent look ahead up to four cells; vertical offsets $\Delta y \in \{-3, \dots, +3\}$ cover above and below. The rationale behind this is that giving it full grid visibility would lead an explosion of possible programs, while this small subset allows for sufficient behaviour exploration while allowing it to learn to react to close threats. Additionally, since the framework stores terrain and enemy data in the same grid, both predicates query the same array and differ only in the value range they compare against: enemy type codes (2–13) versus tile codes.

An earlier version of the grammar used precomputed boolean predicates computed outside the GP tree (`hole_ahead`, `enemy_ahead`, `wall_ahead`). This simplified the search space but also reduced it: the agent could not, for example, distinguish between enemy types or check arbitrary offsets. Though experiments with this grammar offered interesting insights (discussed below), these changes were reverted in order to align with the project statement’s scope.

Terminals. Enemy kind terminals cover all sprite types: Goomba (standard and winged), Red and Green Koopa (standard and winged), Bullet Bill, Spiky (standard and winged), Piranha Flower, and Shell. Tile value terminals are `HARDOBSTACLE` (−10), `BRICK` (16), and `ENEMYOBSTACLE` (20). Action key terminals are `RIGHT`, `LEFT`, `JUMP`, `SPEED`, and `DOWN`.

2.3 Fitness Calculation

Stage 1 The first fitness function uses a very simple pattern:

$$r_t = \Delta x_t + P_{\text{stuck}}(t) \quad (1)$$

Forward progress. $\Delta x_t = \mathbf{1}[x_t - x_{t-1} \geq 0.61]$ awards one point when Mario advances at least 0.61 world units in a step. The binary nature of the function ensures that Mario doesn’t get rewarded for running fast for no substantial gain, walking or speeding is the same (walking is most of the times efficient because it allows for better reasoning and control). The threshold is used so that stationary oscillations near a wall do not accumulate a positive reward, as that would go against the next term of the function.

Stuck penalty. When Mario has made no forward progress for more than 10 consecutive steps (in order to not penalize legitimate pauses) and a solid obstacle occupies the cell directly ahead, a growing penalty is applied:

$$P_{\text{stuck}}(t) = \begin{cases} -10(n_{\text{stuck}} - 10) & \text{if } n_{\text{stuck}} > 10 \text{ and obstacle ahead} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The condition additionally requires that no Piranha Flower is present in the column above Mario. This exclusion was added after discussing how the original penalty, which lacked it, could cause agents to rush into Piranha pipes rather than waiting for the enemy to retract.

Airtime penalty. A behavior that soon became apparent in high-fitness individuals was their tendency to jump constantly. We theorized that agents converged to this behaviour because, due to the simplicity of their grammar, it is very hard to reach a proper ruleset of when jumping is favorable. As such, when an individual stumbles into this strategy in the early generations, it offers a massive jump in fitness, causing it to flood the population, which attempts to optimize it until the end of the run.

In order to counteract this behavior, a penalty proportional to the total air time was initially attempted and iterated over several tests. However, these either failed to prevent the final agent from jumping, or made it impossible for agents to learn to make any progress. This strengthened our belief that, with the current grammar, agents were only given the space to understand that the air is usually safer than the ground (at least in the first three difficulties). It is further supported by the fact that we were able to evolve Marios that did not display this behavior but still jumped over enemies and holes before removing those complex perceptions from our grammar.

Episode average. For the experiments run for the results section, each individual is evaluated over a seed pool $\mathcal{S}$ of 5 level seeds and 3 consecutive difficulty levels starting at base difficulty 0. A win bonus ($W = 10,000$) is added when the level is cleared. Fitness is the mean reward across all $|\mathcal{S}| \times 3$ episodes:

$$F = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \sum_{d=d_{\text{base}}}^{d_{\text{base}}+2} \left(\sum_t r_t + W \cdot \mathbf{1}[\text{win}_{s,d}] + L \cdot \mathbf{1}[\text{lose}_{s,d}] \right) \quad (3)$$

Stage 2 This stage adds a kill reward to the reward function:

$$r_t = \Delta x_t + P_{\text{stuck}}(t) + R_{\text{kill}}(t) \quad (4)$$

Kill reward. **[TODO: update value when tests are complete]** Each confirmed kill contributes 10,000 points:

$$R_{\text{kill}}(t) = 10,000 \hat{k}_t \quad (5)$$

Kill detection compares the enemy list between consecutive frames, counting reductions in enemy count while handling the exceptions that occur with enemies from the first three difficulties: Koopa-to-Shell transitions (a Koopa count drop paired with a Shell count rise maintains the enemy list size, but is counted as a kill); enemies falling into pits (filtered by a y position threshold); enemies scrolling off-screen as Mario advances (filtered by horizontal distance); and Piranha Flowers retracting into pipes (detects plausible retractions). A disappearing enemy is counted as a kill only if Mario’s vertical velocity at that step is consistent with a stomp bounce, meaning a sign reversal or a deceleration pattern that matches rebounding off the top of an enemy.

This replaced an earlier heuristic that counted kills whenever the number of enemies within a small radius around Mario decreased between frames. That approach produced frequent false positives in crowded sections, inflating the reward signal and leading agents to stand still near enemy clusters rather than completing the level.

Coin reward. **[TODO: write description when tests are complete]**

Table 1. Evolutionary algorithm parameters used in the main experiments.

Parameter	Value
Population size	100
Generations	300
Crossover rate (p_c)	0.6
Mutation rate (p_m)	1.0 $\rightarrow$ 0.4 (linear over training)
Crossover operator	One-point (<code>gp.cxOnePoint</code>)
Mutation operator	Uniform subtree (<code>gp.mutUniform</code>)
Selection	Double Tournament
Fitness tournament size	2 $\rightarrow$ 7 (linear over training)
Parsimony tournament size	1.7
Max tree height	10
Elitism	Top-1 (Hall of Fame)

2.4 Evolutionary Algorithm

Table 1 lists the final parameter configuration.

Selection and bloat control. We use Double Tournament selection [4], which runs two nested tournaments: the outer selects based on fitness (with a size that grows from 2 to 7 over training), and the inner selects based on tree size with a fractional tournament of size 1.7, preferring shorter programs when fitness is similar. This avoids adding a size term to the fitness function directly, which requires careful weighting and tends to penalize legitimately complex programs that need more nodes to express a useful behavior.

Crossover and mutation. One-point crossover selects a random node in each parent and swaps the corresponding subtrees. If the resulting offspring exceeds height 10, the operation is reverted and the parent is preserved. Uniform subtree mutation selects a random node and replaces its subtree with a freshly grown random subtree (depth 2 - 6), so the algorithm introduces genuinely new code fragments rather than only recombining existing material.

The mutation rate decreases from 1.0 to 0.4 over training. Preliminary experiments with fixed rates struggled with keeping an appropriate balance between exploration and exploitation at different points in the experiment. The dynamic schedule addresses both: high mutation early for exploration, lower mutation late for exploitation. The rationale behind the fitness tournament size is the same: it grows linearly so that early generations explore broadly under low selection pressure and later generations converge under stronger pressure once the search has found promising regions.

Elitism. The single best individual across all generations, stored in a Hall of Fame, is copied directly into the next generation at each step. This ensures the best fitness found is never lost to a bad crossover or mutation.

Parallelism. The provided evaluation infrastructure distributes fitness evaluations across 10 worker processes, each holding a persistent connection to its own simulator instance. We extended it to support multi-seed pools, per-generation difficulty scheduling, and lazy re-evaluation: given the simulation’s deterministic nature, only individuals whose fitness was invalidated by a genetic operator need to be re-evaluated.

3 Experimental Design

3.1 Evaluation Protocol

All individuals within a generation are evaluated on the same 5 randomly drawn level seeds, so that fitness values are directly comparable within a generation. Using different seeds per individual, as we did in early experiments, introduced noise that made selection unreliable: an individual with higher fitness might simply have faced an easier level rather than exhibiting a better strategy, which led to the population stagnating on a few lucky seeds rather than improving.

To prevent overfitting to specific levels, the seed pool is rotated every 10 generations by drawing a fresh set from a universe of 1,000 seeds. At each rotation, all population fitnesses are invalidated and re-evaluated under the new pool; the Hall of Fame champion’s fitness is also reset, so its score always reflects the current pool rather than a comparison made against an earlier one. A checkpoint is saved at each rotation. At the end of training, all checkpoints are re-evaluated on a fixed set of 500 held-out seeds across the three training difficulties (1,500 episodes per checkpoint). The Hall of Fame champion from the checkpoint with the highest overall win rate on this held-out set is selected as the final evolved agent. This re-evaluation step is necessary because the Hall of Fame score stored in each checkpoint reflects fitness under whichever seed pool was active during that rotation, making raw fitness values across checkpoints incomparable without a shared evaluation basis.

For seed pool rotation, periods of 5, 10, and 20 generations were compared, with the middle option achieving overall better results. Rotating every 5 generations did not give the population enough time to exploit each seed pool before it changed, slowing convergence. Rotating every 20 generations allowed the population to overfit to specific level seeds, with agents that scored well on the current pool but failed to generalize when the pool changed. Rotating every 10 generations provided a good balance: enough stability for selection to act meaningfully, and enough change to prevent overfitting.

The base difficulty d_{base} is fixed at 0 throughout training, so agents are always evaluated on difficulties 0, 1, and 2 simultaneously. An earlier design explored a curriculum that gradually shifted d_{base} upward over generations. This was abandoned when training on all three difficulties from the start produced better-generalizing agents, likely because the easier difficulty levels provide a reliable learning signal even when the population is still immature.

Finally, we chose to run 300-generation runs to fully explore the potential of the evolutionary approach, even though there were clear signs of stagnation after about 100 generations. 200-individual and 100-individual populations were compared, with the latter configuration producing comparable completion rates at half the wall-clock cost, and was adopted as the default.

3.2 Comparative Baseline

We compare evolved champions against a random search baseline to isolate the contribution of the evolutionary process from the expressive power of the grammar. We run five independent training experiments, each with a different random seed. For each run, the champion is selected by re-evaluating all checkpoints as described above. The baseline for each run is a single program drawn uniformly at random from the same grammar using the matching seed, with no selection or recombination applied. Both agents are then evaluated on 5,000 held-out level seeds across three difficulty levels (15,000 episodes per agent), and we report mean and standard deviation across the five runs.

As a further comparison, we replicated the full experiment with Mario initialised as *big Mario* carrying a fire flower (hereafter *Fire Mario*). The grammar was extended with a `fire` action, but all training and evaluation parameters—population size, generations, seed-pool rotation, checkpoint selection, and held-out set size—remained identical. Descriptive statistics for Fire Mario are presented together with the normal Mario results in Table ?? . Formal significance testing was reserved for the primary (normal-Mario) comparison.

3.3 Statistical Analysis

Since both the evolutionary process and the level generator are stochastic, a single run is insufficient to draw reliable conclusions [3]. We run five independent training seeds and evaluate the resulting champions and their matched baselines on 5,000 held-out level seeds across three difficulties, giving 15,000 episodes per agent. We report mean and standard deviation across the five runs for three metrics:

- **Completion rate:** percentage of episodes in which the level is cleared before death or timeout.
- **Max distance:** mean maximum x -position reached in episodes that did not result in a level clear.
- **Enemies killed:** mean number of confirmed kills per episode.
- **Coins collected:** mean number of coins gathered per episode.

To quantify the reliability of the observed advantage of the evolved normal Mario agents over the random baseline, we conducted paired-sample t -tests on completion rates, enemies killed, and coins collected. The results indicated that

[TODO: insert whether differences were statistically significant and in which metrics].

4 Results and Analysis

4.1 Fitness Convergence

Figures 1, 2, and 3 plot the win rate of the best individual (Hall of Fame) at each checkpoint, evaluated on 100 held-out level seeds across the three training difficulties, giving 300 episodes per data point.

In Figure 1 (population 100, normal mode), at around generation 10 the win rate jumps from near zero to approximately 31%, with difficulty-0 reaching 65% in the same checkpoint. From that point the overall win rate stabilizes in the 38–40% range for the remaining 290 generations, with difficulty-0 peaking near 73% around generation 150. Performance degrades with difficulty: difficulty-1 settles at roughly 20–33% and difficulty-2 at 9–18%, reflecting the increasing obstacle density and enemy variety at higher settings. The gap between the difficulty-0 line and the overall line shows that the agent learned to clear open, low-obstacle levels reliably but struggled to generalize to adversarial layouts. Seed rotation events are visible as short-lived dips at multiples of 10 generations; each dip is followed by recovery within the next rotation period, confirming that the population’s strategy is general enough to survive seed-pool changes rather than collapsing on each rotation.

Figure 2 (population 200, normal mode) shows the same qualitative trajectory with a 40-generation delay: the best agent’s overall win rate remains near zero until generation 50, then converges rapidly to the same 38–40% plateau. The delay is likely due to the larger initial population requiring more episodes before a first viable run-and-jump strategy emerges and propagates. Final performance is nearly identical to the 100-population run, which supports choosing the smaller configuration as the default: the same quality is reached at roughly half the computational cost.

Figure 3 (population 200, fire mode) shows markedly higher win rates. Difficulty-0 reaches near 100% from generation 55 onward, difficulty-1 stabilizes between 60–75%, and difficulty-2 ranges from 34–60%, giving an overall win rate of 65–70%. Fire Mario eliminates most enemies with projectiles rather than stomps, which removes the need for precise vertical positioning during kills. Since the grammar and fitness function are identical, the improvement is entirely attributable to the task becoming easier to navigate, , showing the ceiling that normal-mode enemy encounters impose.

4.2 Evolved Agent vs. Random Search Baseline

Table 2 compares the champion agent against the random search baseline across $N = 5$ held-out seeds and 3 difficulty levels.

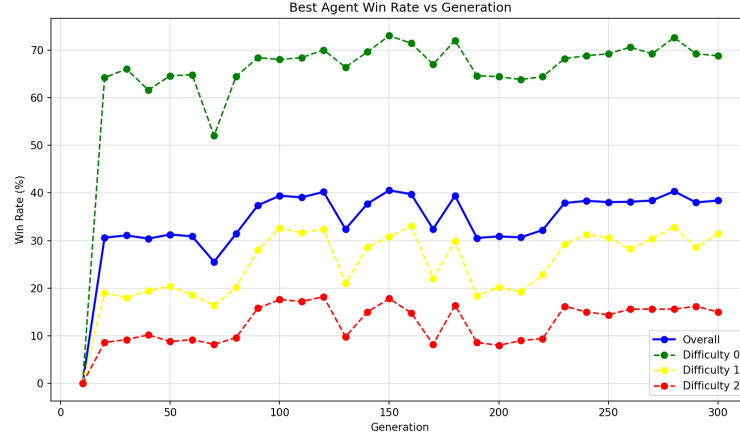


Fig. 1. Win rate vs. generation, population size 100, normal Mario mode.

Table 2. Evolved champion vs. random search baseline ($N = 5$ held-out seeds $\times$ 3 difficulties, mean $\pm$ std). **[TODO: Fill in all values.]**

Metric	Random Search (Avg $\pm$ Std)	Evolved Agent (Avg $\pm$ Std)	Improvement
Levels Cleared (%)	[TODO: X]	[TODO: Y]	[TODO: +Z%]
Max Distance (px)	[TODO: X]	[TODO: Y]	[TODO: +Z%]
Enemies Killed	[TODO: X]	[TODO: Y]	[TODO: +Z%]

[TODO: Comment on the table. Describe the qualitative difference in behavior: the random agent will jump and move without a coordinated strategy, while the evolved agent should show consistent forward movement and at minimum a learned response to walls and pits. Discuss whether the improvement is meaningful given the observed standard deviations, or whether the variance is too large to draw firm conclusions.]

4.3 Qualitative Analysis of Evolved Behavior

The best individual found in the 100-population normal-mode run (Hall of Fame score: 51114) is shown below.

```
def corre(action, landscape, enemies, can_jump,
          on_ground, mario_pos, Mario, Sprite, **kwargs):

    if (enemies[11+-1][11+1] < 8):
        if (not (landscape[11+1][11+3] > 16)):
            if (enemies[11+-1][11+1] < 12):
                if (not (on_ground or on_ground)):
                    action[Mario.KEY_RIGHT] = 1
```

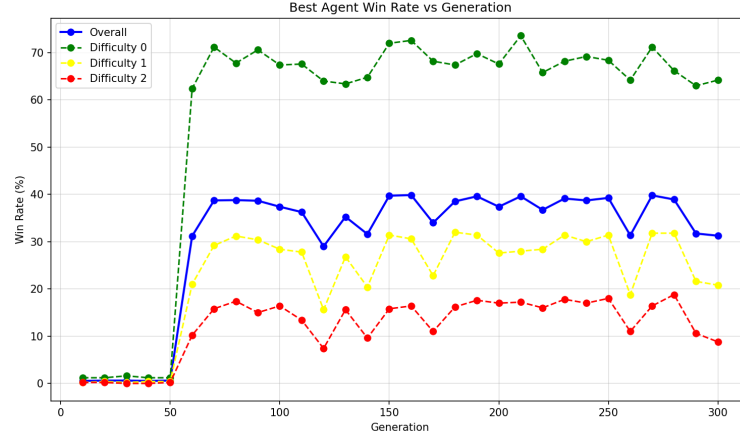


Fig. 2. Win rate vs. generation, population size 200, normal Mario mode.

```

else:
    action[Mario.KEY_SPEED] = 1
else:
    action[Mario.KEY_SPEED] = 1
if ((not (on_ground or ((enemies[11+0][11+2] < 13)
    and ((enemies[11+0][11+3] == 13) or on_ground))))
    or can_jump):
    action[Mario.KEY_JUMP] = 1
else:
    if (landscape[11+0][11+3] < 16):
        if (not (enemies[11+2][11+0] == 8)):
            if can_jump:
                if (enemies[11+3][11+0] == 8):
                    if can_jump:
                        action[Mario.KEY_SPEED] = 0
                    else:
                        if on_ground:
                            action[Mario.KEY_LEFT] = 1
                else:
                    if (can_jump and on_ground):
                        if on_ground:
                            action[Mario.KEY_RIGHT] = 1
                        else:
                            if can_jump:
                                action[Mario.KEY_LEFT] = 1
                            else:
                                action[Mario.KEY_RIGHT] = 1
            else:
                if can_jump:
                    if (enemies[11+3][11+3] < 8):

```

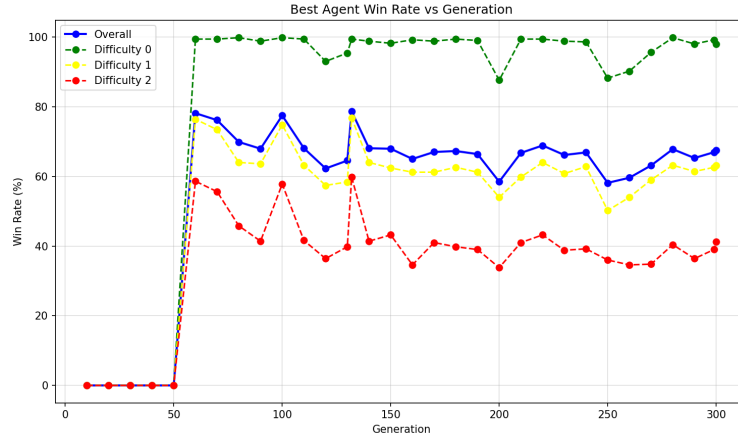


Fig. 3. Win rate vs. generation, population size 200, fire Mario mode.

```

if can_jump:
    if can_jump:
        action[Mario.KEY_DOWN] = 0
    else:
        action[Mario.KEY_LEFT] = 0
else:
    if can_jump:
        action[Mario.KEY_JUMP] = 0

```

The program consists of two sequential statements. The first block handles the speed decision: it checks for an enemy of type < 8 (Goomba or empty) one cell above and one cell ahead, and, if no tile with code > 16 (i.e., no enemy obstacle) blocks the path three columns ahead at ground level, sets `SPEED`. This increases forward momentum whenever a low-threat enemy is nearby and the terrain ahead is passable.

The second block encodes the dominant behavior. Its condition, `(not (on_ground or ...)) or can_jump`, simplifies to `can_jump` on almost every ground-contact frame, meaning the agent jumps on every step it is able to. The result is a controller that runs right and jumps continuously, with the speed modulation in the first block providing the only enemy-sensitive adjustment.

The tree contains several introns. `on_ground` or `on_ground` evaluates identically to `on_ground` but occupies two leaf nodes and one OR node. The else branch of the jump condition (lines 18–44) covers the brief window when Mario is grounded but `can_jump` is false; since this window lasts only a few frames per landing, most of the nested conditions in that branch are rarely reached during a normal episode. Parsimony pressure kept the tree from growing further but

did not eliminate these segments, since their size contribution was small enough to survive the parsimony tournament.

The agent’s primary failure mode is the absence of spatial lookahead in the jump decision: the jump fires based on `can_jump` alone, without checking whether a pit or an obstacle lies ahead. On higher-difficulty levels with more pits and denser enemy placement, this causes the agent to launch itself into gaps it could have avoided, which accounts for the sharp drop in win rate from difficulty 0 to difficulty 2.

5 Conclusion

[TODO: Write after results are confirmed. Structure: (1) one paragraph restating the approach and the two objectives. (2) The most impactful design choices, with results woven in, e.g., “Switching to the type-aware kill detector raised confirmed kills per episode from X to Y; the radius heuristic had attributed those same kills to off-screen scrolling.” (3) Limitations: fixed-arity grammar with no loops or arithmetic, evaluation cost constraining population size, sensitivity to kill-reward weighting relative to the progress signal. (4) Future directions: extending the grammar with loops and numeric comparisons, island models to maintain diversity across subpopulations, and adaptive operator tuning via CMA-ES.]

References

1. Fortin, F.A., De Rainville, F.M., Gardner, M.A., Parizeau, M., Gagné, C.: DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research* **13**, 2171–2175 (2012)
2. Koza, J.R.: *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA (1992)
3. Lourenço, N.: How to evaluate and compare bio-inspired algorithms (2025), lecture slides, Nature-Inspired Artificial Intelligence, Universidade de Coimbra
4. Luke, S., Panait, L.: A comparison of bloat control methods for genetic programming. *Evolutionary Computation* **14**(3), 309–344 (2006). <https://doi.org/10.1162/evco.2006.14.3.309>
5. Togelius, J., Karakovskiy, S., Koutník, J., Schmidhuber, J.: Super Mario evolution. In: *Proceedings of the IEEE Symposium on Computational Intelligence and Games (CIG)*. pp. 156–161. IEEE (2009). <https://doi.org/10.1109/CIG.2009.5286481>